Tests-

Sanity check — recompute overall MAE, RMSE, RMSE/MAE ratio from the row-level data, confirm it matches your original ~2.4 ratio (makes sure we reloaded everything correctly).

1. Error concentration test — sort rows by squared error descending; compute what % of total squared error comes from the top 1%, 5%, and 10% of rows. This tells us how "outlier-driven" the RMSE inflation is.
2. Promo vs non-promo breakdown — compare MAE/RMSE/ratio on onpromotion=True days vs False days.
3. Per-store breakdown — group by store_nbr, compute RMSE/MAE ratio per store, find which stores are worst.
4. Per-item-family breakdown — same, grouped by item family/class.
5. Volume-tier breakdown — bin series into low/medium/high average sales, compare error metrics per tier (checks if intermittent/sparse series are the culprit).
6. Error distribution visualization — histogram of signed errors (not absolute) to see tail shape.
7. Worst-case inspection — plot actual vs. predicted for the top 20-30 worst predictions over their 7-day horizon, to eyeball what's going wrong (spike, data anomaly, etc.).
8. Conclusion — written summary of which factor(s) explain most of the RMSE/MAE gap, based on 2-8.

Report-
RMSE/MAE Diagnostic Report — Champion TFT Model

Background

Your project guide flagged that while MAE (12.387 units) is reasonable, RMSE (29.542 units) is roughly **2.39x** higher — well above the ~1.25 ratio expected for well-behaved, Gaussian-distributed forecast errors. This report documents the diagnostic process used to investigate the cause, on the Champion TFT model's test set (22,659 row-level predictions, reshaped from tft_test_forecasts.csv, covering Aug 1–15, 2017, 400 store-item pairs, 7-day horizon).

What We Tested

1. **Sanity check** — recomputed MAE/RMSE/ratio from row-level data to confirm it matched the originally reported metrics.
2. **Error concentration test** — sorted all predictions by squared error and measured what fraction of total squared error came from the top 1%, 5%, and 10% of rows.
3. **Worst-case inspection** — manually examined the 20 highest-error predictions to look for patterns (same stores/items repeating, data anomalies, etc.).
4. **Store-item pair identification** — counted how many *unique* store-item pairs were responsible for the top 1% of errors.

5. **Metadata cross-reference** — joined the worst-offending pairs against selected_pairs.csv to check their average_sales, demand_bin, and family classification.
6. **Bias direction check** — measured mean error and % of under- vs. over-predictions specifically for the worst-offending pairs, to see if the model has a systematic directional bias or just high variance.
7. **Horizon-day breakdown** — checked whether error for these pairs worsens as the forecast horizon extends (Day+1 vs Day+7).

What We Found

Finding	**Result**
Overall RMSE/MAE ratio	2.39 (vs. ~1.25 expected for Gaussian errors)
Squared error from top 1% of rows	62.1%
Squared error from top 5% of rows	84.3%
Squared error from top 10% of rows	91.7%
Unique store-item pairs in top 1% worst errors	20 (out of ~400)
Rows belonging to these 20 pairs	1,204 (5.3% of all rows)
Squared error contributed by these 20 pairs	**74.5%**
demand_bin of all 20 pairs	very_high (19/20) — the highest-volume tier in the dataset
Mean error for these pairs (true − pred)	+8.6 (near zero — slight underprediction on average)
% of rows underpredicted vs overpredicted for these pairs	47.3% underpredicted / 52.7% overpredicted — roughly balanced
RMSE/MAE ratio *within* these 20 pairs alone	1.47–1.67 — much closer to the Gaussian benchmark
Horizon-day degradation for these pairs	Flat/stable across Day+1 to Day+7 — no progressive worsening

In plain terms: the model isn't broadly unreliable. A small, identifiable group of 20 store-item pairs — your highest-selling SKUs (avg. 15–497 units/day) — generate three-quarters of the total squared error, even though they make up only 5.3% of the test rows. Within that group, the model doesn't have a consistent directional bias (it's not always under- or over-shooting); it has **high day-to-day variance**, which squared-error metrics punish disproportionately simply because the absolute swings are naturally bigger for high-volume items (a 50-unit miss on an item averaging 300 units/day is "normal" relative noise, but contributes a huge number to RMSE).

This matches known behavior in retail demand forecasting: a small number of fast-moving SKUs dominate squared-error metrics, while the bulk of the catalog (slow/medium movers) forecasts cleanly — which is exactly why your MAE stays low while RMSE balloons.

What Can Be Done to Fix This

1. Segment-aware loss weighting (moderate effort, no architecture change)
Give the high-volume demand_bin = very_high series higher weight in the Huber loss during retraining, so the model is penalized more for missing them. Risk: may slightly hurt performance on the other 95% of series. Needs one retrain, not an architecture redesign.

2. Per-tier modeling or scaling (moderate-high effort)
Train the high-volume tier (or just these 20 pairs) with a separate model/head, or apply tier-specific feature scaling instead of one global scaler — since target_sales_scaler.joblib applying one log1p + StandardScaler globally may compress large values' gradient signal during training.

3. Quantile/probabilistic forecasting (highest effort, most defensible fix)
Predict P10/P50/P90 instead of a point forecast. This doesn't eliminate the variance, but it gives your downstream PSO-based inventory optimization the uncertainty band directly — which is arguably more useful than a lower RMSE, since safety stock calculations need a sense of forecast *uncertainty*, not just a point estimate. This is the standard fix in the forecasting literature for exactly this kind of heavy-tailed, high-volume-driven error pattern, but it requires retraining with a different loss function (pinball/quantile loss) and more careful evaluation.